Name : Ishan Manekar

Class : A_A3_45

In [2]:
```
import pandas as pd
import numpy as np
```

In [3]:
```
df = pd.read_csv("enjoy.csv")
```

In [4]:
```
df.head()
```

Out[4]:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes
4	Sunny	Cold	Normal	Weak	Warm	Same	No

In [5]:
```
df.tail()
```

Out[5]:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
5	Cloudy	Warm	Normal	Weak	Warm	Same	No
6	Sunny	Warm	Normal	Weak	Cool	Same	Yes
7	Rainy	Warm	High	Weak	Cool	Change	No
8	Sunny	Warm	Normal	Strong	Cool	Same	Yes
9	Sunny	Warm	High	Weak	Warm	Same	Yes

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Sky         10 non-null     object
 1   AirTemp     10 non-null     object
 2   Humidity    10 non-null     object
 3   Wind        10 non-null     object
 4   Water       10 non-null     object
 5   Forecast    10 non-null     object
 6   EnjoySport  10 non-null     object
dtypes: object(7)
memory usage: 692.0+ bytes
```

In [7]:
```
df.describe()
```

Out[7]:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
count	10	10	10	10	10	10	10
unique	3	2	2	2	2	2	2
top	Sunny	Warm	Normal	Strong	Warm	Same	Yes
freq	7	8	5	5	6	7	6

In [8]:
```
df.isnull()
```

Out[8]:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False

In [25]:
```
X = df.iloc[:,:-1].values

y = df.iloc[:,-1].values
```

In [27]:
```
def find_s(X,y):
    hypothesis =None
    print("Initial Hypothesis: ", hypothesis)
    for i in range(len(X)):
        if y[i] == "Yes":
            if hypothesis is None:
                hypothesis = X[i].copy()
            else :
                for j in range(len(hypothesis)):
                    if hypothesis[j] != X[i][j]:
                        hypothesis[j] = "?"

        print(f"\n After training example {i+1}")
        print(hypothesis)
    return hypothesis

final_hypothesis = find_s(X,y)

print("\n Final Hypothesis")
print(final_hypothesis)
```

```
Initial Hypothesis:  None

 After training example 1
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

 After training example 2
['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']

 After training example 3
['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']

 After training example 4
['Sunny' 'Warm' '?' 'Strong' '?' '?']

 After training example 5
['Sunny' 'Warm' '?' 'Strong' '?' '?']

 After training example 6
['Sunny' 'Warm' '?' 'Strong' '?' '?']

 After training example 7
['Sunny' 'Warm' '?' '?' '?' '?']

 After training example 8
['Sunny' 'Warm' '?' '?' '?' '?']

 After training example 9
['Sunny' 'Warm' '?' '?' '?' '?']

 After training example 10
['Sunny' 'Warm' '?' '?' '?' '?']

 Final Hypothesis
['Sunny' 'Warm' '?' '?' '?' '?']
```

In [29]:
```
## Candidate Elimination Algorithm
```

In [31]:
```
def candidate_elimination(concepts, target):
    specific = concepts[0].copy()
    general =[["?" for i in range(len(specific))]]
    print("\n Initial S: ", specific)
    print("Initial G :", general)
    for i, h in enumerate(concepts):
        if target[i] == "Yes":
            for x in range(len(specific)):
                if h[x] != specific[x]:
                    specific[x] = "?"
        else:
            for x in range(len(specific)):
                if h[x] !=specific[x]:
                    general.append(["?" if j!=x else specific[x] for j in ra
        print("\n Example", i+1)
        print("S = ", specific)
        print("G = ", general)
    return specific, general

S, G = candidate_elimination(X,y)

print("\n Final Specific Boundary")

print(S)

print("\n Final General Boundary")

for g in G:
    print(G)
```

```
 Initial S:  ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
Initial G : [['?', '?', '?', '?', '?', '?']]

 Example 1
S =  ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
G =  [['?', '?', '?', '?', '?', '?']]

 Example 2
S =  ['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']
G =  [['?', '?', '?', '?', '?', '?']]

 Example 3
S =  ['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']
G =  [['?', '?', '?', '?', '?', '?'], ['Sunny', '?', '?', '?', '?', '?'],
['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?',
'?', '?', '?', '?', 'Same']]

 Example 4
```

In [ ]: